

LAPORAN PRAKTIKUM
STRUKTUR DATA
ALGORITMA SORTING PADA JAVA

Diususn Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 7 dengan judul “Algoritma Sorting pada Java” ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai bagaimana cara kerja beberapa algoritma sorting (bubble sort, insertion sort dan selection sort) dan menerapkannya untuk membuat sebuah program sorting sederhana.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Averro yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami algoritma sorting pada java

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Buble Sort	2
2.1.1 Penjelasan Program.....	2
2.1.2 Kode Program dan Output	2
2.2 Insertion Sort.....	4
2.2.1 Penjelasan Program.....	4
2.2.2 Kode Program dan Output	4
2.3 Selection Sort	6
2.3.1 Penjelasan Program.....	6
2.3.2 Kode Program dan Output	6
2.4 Insertion Sort GUI.....	8
2.4.1 Penjelasan Program.....	8
2.4.2 Kode Program dan Output	9
BAB III PENUTUP	13
TUGAS PRAKTIKUM.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (sorting) merupakan fundamental yang sangat krusial karena efisiensi sebuah sistem atau pencarian informasi sering kali bergantung pada bagaimana data tersebut diorganisasikan. Algoritma dasar seperti Bubble Sort, Insertion Sort, dan Selection Sort merupakan fondasi awal yang wajib dipahami untuk mempelajari kompleksitas algoritma dan manajemen memori.

Penerapan antarmuka grafis atau GUI (Graphical User Interface) menggunakan library Java Swing. Dengan menggabungkan pemahaman logika kode murni dan visualisasi berbasis GUI, dapat mengamati bagaimana elemen data bertukar tempat, bergeser, dan diposisikan pada setiap iterasi.

1.2 Tujuan Praktikum

- Memahami mekanisme kerja, logika dasar, serta perbedaan karakteristik dari algoritma pengurutan Bubble Sort, Insertion Sort, dan Selection Sort dalam bahasa pemrograman Java.
- Menganalisis proses iterasi, perbandingan (comparison), dan pertukaran data (swap) yang terjadi pada masing-masing algoritma pengurutan.
- Mengimplementasikan konsep pemrograman berbasis grafis menggunakan Java Swing untuk memvisualisasikan langkah demi langkah eksekusi algoritma sorting secara interaktif.

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah dapat mengasah kemampuan logika pemrograman dan analisis kompleksitas, serta mempermudah pemahaman visual tentang bagaimana data di memori dimanipulasi oleh algoritma pengurutan.

BAB II

PEMBAHASAN

2.1 Buble Sort

2.1.1 Penjelasan Program

Program ini mengurutkan array dengan membandingkan dua elemen yang bersebelahan secara berulang dari awal hingga akhir, lalu menukarnya jika elemen kiri lebih besar dari elemen kanan. Melalui proses ini, nilai terbesar perlahan akan pindah ke posisi paling kanan pada setiap putaran luar, dengan menampilkan di dalam blok pertukaran untuk melacak setiap detail pergeseran angka secara langsung saat proses pengurutan terjadi.

2.1.2 Kode Program dan Output

```
public class BubleSort_2511532008 {
    public static void bubbleSort_2008(int[] arr_2008) {
        int n_2008 = arr_2008.length;
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            for (int j_2008 = 0; j_2008 < n_2008 - i_2008 - 1; j_2008++) {
                if (arr_2008[j_2008] > arr_2008[j_2008 + 1]) {
                    int temp_2008 = arr_2008[j_2008];
                    arr_2008[j_2008] = arr_2008[j_2008 + 1];
                    arr_2008[j_2008 + 1] = temp_2008;
                    System.out.println("data:"+arr_2008[j_2008]+" "+arr_2008[j_2008+1]);
                }
            }
        }
    }
    public static void main(String[] args) {
        int arr_2008[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2008 = arr_2008.length;
        System.out.print("array yang belum terurut:");
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println("");
        bubbleSort_2008(arr_2008);
        System.out.print("array yang terurut menggunakan BubleSort:");
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println("");
    }
}
```

Gambar 2.1

```
array yang belum terurut : 23 78 45 8 32 56 1  
data : 45 78  
data : 8 78  
data : 32 78  
data : 56 78  
data : 1 78  
data : 8 45  
data : 32 45  
data : 1 56  
data : 8 23  
data : 1 45  
data : 1 32  
data : 1 23  
data : 1 8  
array yang terurut menggunakan BubleSort : 1 8 23 32 45 56 78
```

Gambar 2.2

2.2 Insertion Sort

2.2.1 Penjelasan Program

Program ini bekerja seperti cara manusia mengurutkan kartu di tangan, yaitu dengan membagi array secara imajiner menjadi bagian yang sudah terurut dan belum terurut. Dimulai dari elemen kedua, setiap angka diambil sebagai kunci (key) lalu disisipkan ke posisi yang tepat di bagian kiri yang sudah terurut dengan cara menggeser elemen-elemen yang lebih besar ke kanan.

2.2.2 Kode Program dan Output

```
public class InsertionSort_2511532008 {
    public static void insertionSort_2008 (int [] arr_2008) {
        int n = arr_2008.length;
        for (int i =1; i< n; i++) {
            int key_2008 = arr_2008[i];
            int j = i - 1;
            while (j >= 0 && arr_2008[j] > key_2008) {
                arr_2008 [j + 1] = arr_2008[j];
                j--;
            }
            arr_2008[j + 1] = key_2008;
        }
    }
    public static void main (String []args) {

        int arr_2008[] = {23 , 78,45,8,32,56,1};
        int n = arr_2008.length;

        System.out.printf("Array yg belum terurut : \n");
        for (int i = 0; i < n; i++)
            System.out.print(arr_2008[i] + " ");
        System.out.println("");
        insertionSort_2008(arr_2008);
        System.out.printf("Array yg sudah terurut : \n");
        for (int i = 0; i <n; i++)
            System.out.print(arr_2008[i]+" ");
        System.out.println("");
    }
}
```

Gambar 2.3

```
Array yg belum terurut :  
23 78 45 8 32 56 1  
Array yg sudah terurut :  
1 8 23 32 45 56 78
```

Gambar 2.4

2.3 Selection Sort

2.3.1 Penjelasan Program

Program ini memproses pengurutan dengan cara mencari elemen dengan nilai terkecil dari seluruh bagian array yang belum terurut pada setiap putaran luar. Setelah posisi elemen terkecil ditemukan melalui perulangan dalam, program hanya melakukan satu kali pertukaran posisi untuk menempatkan elemen terkecil tersebut ke indeks awal yang menjadi target putaran. Metode ini sangat efisien dalam meminimalkan jumlah operasi pertukaran (swap) data di memori jika dibandingkan dengan Bubble Sort.

2.3.2 Kode Program dan Output

```
public class SelectionSort_2511532008 {
    public static void selectionSort_2008(int[] arr_2008) {
        int n_2008 = arr_2008.length;
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            int minIndex_2008 = i_2008;
            for (int j_2008 = i_2008 + 1; j_2008 < n_2008; j_2008++) {
                if (arr_2008[j_2008] < arr_2008[minIndex_2008]) {
                    minIndex_2008 = j_2008;
                }
            }
            int temp_2008 = arr_2008[i_2008];
            arr_2008[i_2008] = arr_2008[minIndex_2008];
            arr_2008[minIndex_2008] = temp_2008;
        }
    }
    public static void main(String[] args) {
        int arr_2008[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2008 = arr_2008.length;

        System.out.printf("array yang belum terurut:\n");
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println("");
        selectionSort_2008(arr_2008);
        System.out.printf("array yang terurut:\n");
        for (int i_2008 = 0; i_2008 < n_2008; i_2008++) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println("");
    }
}
```

Gambar 2.5

```
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

Gambar 2.6

2.4 Insertion Sort GUI

2.4.1 Penjelasan Program

Program ini merupakan visualisasi interaktif berbasis Java Swing yang mendemonstrasikan proses pengurutan Insertion Sort langkah demi langkah kepada pengguna. Melalui antarmuka grafis, pengguna dapat memasukkan deretan angka acak yang dipisahkan oleh koma, yang kemudian diubah menjadi kotak-kotak label visual di layar serta mengaktifkan tombol kontrol utama. Setiap kali tombol "Langkah Selanjutnya" diklik, sistem mengeksekusi satu iterasi luar dari algoritma Insertion Sort—mengambil satu angka kunci (key), menggeser elemen yang lebih besar di sebelah kirinya, menempatkan angka tersebut pada posisi yang tepat, lalu memperbarui tampilan kotak angka sekaligus mencetak log pencatatan proses secara mendetail pada area teks di sisi kanan hingga pengurutan selesai sepenuhnya.

2.4.2 Kode Program dan Output

```
import java.awt.BorderLayout;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import java.awt.Font;
import javax.swing.JScrollPane;
import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.SwingConstants;
import java.awt.Color;
import java.awt.Dimension;
import javax.swing.SwingUtilities;

public class InsertionSortGUI_2511532008 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_2008;
    private JLabel[] labelArray_2008;
    private JButton stepButton_2008, resetButton_2008, setButton_2008;
    private JTextField inputField_2008;
    private JPanel panelArray_2008;
    private JTextArea stepArea_2008;
    private JPanel contentPane;

    private int i_2008 = 1,j_2008;
    private boolean sorting_2008 = false;
    private int stepCount_2008;
```

Gambar 2.7

```

public InsertionSortGUI_2511532008() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    JPanel inputPanel_2008 = new JPanel(new FlowLayout());
    inputField_2008 = new JTextField(30);
    setButton_2008 = new JButton("Set Array");
    inputPanel_2008.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
    inputPanel_2008.add(inputField_2008);
    inputPanel_2008.add(setButton_2008);

    panelArray_2008 = new JPanel();
    panelArray_2008.setLayout(new FlowLayout());

    JPanel controlPanel_2008 = new JPanel();
    stepButton_2008 = new JButton("Langkah Selanjutnya");
    resetButton_2008 = new JButton("Reset");
    stepButton_2008.setEnabled(false);
    controlPanel_2008.add(stepButton_2008);
    controlPanel_2008.add(resetButton_2008);

    stepArea_2008 = new JTextArea(8, 60);
    stepArea_2008.setEditable(false);
    stepArea_2008.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_2008 = new JScrollPane(stepArea_2008);

    add(inputPanel_2008, BorderLayout.NORTH);
    add(panelArray_2008, BorderLayout.CENTER);
    add(controlPanel_2008, BorderLayout.SOUTH);
    add(scrollPane_2008, BorderLayout.EAST);

    setButton_2008.addActionListener(e_2008 -> setArrayFromInput_2008());
    stepButton_2008.addActionListener(e_2008 -> performStep_2008());
    resetButton_2008.addActionListener(e_2008 -> reset_2008());
}

```

Gambar 2.8

```

private void setArrayFromInput_2008() {
    String text_2008 = inputField_2008.getText().trim();
    if (text_2008.isEmpty()) return;
    String[] parts_2008 = text_2008.split(",");
    array_2008 = new int[parts_2008.length];
    try {
        for (int k_2008 = 0; k_2008 < parts_2008.length; k_2008++) {
            array_2008[k_2008] = Integer.parseInt(parts_2008[k_2008].trim());
        }
    } catch (NumberFormatException e_2008) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan \n"
            + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_2008 = 1;
    stepCount_2008 = 1;
    sorting_2008 = true;
    stepButton_2008.setEnabled(true);
    stepArea_2008.setText("");
    panelArray_2008.removeAll();
    labelArray_2008 = new JLabel[array_2008.length];
    for (int k_2008 = 0; k_2008 < array_2008.length; k_2008++) {
        labelArray_2008[k_2008] = new JLabel(String.valueOf(array_2008[k_2008]));
        labelArray_2008[k_2008].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2008[k_2008].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_2008[k_2008].setPreferredSize(new Dimension(50, 50));
        labelArray_2008[k_2008].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2008.add(labelArray_2008[k_2008]);
    }
    panelArray_2008.revalidate();
    panelArray_2008.repaint();
}

```

Gambar 2.9

```

private void performStep_2008() {
    if (i_2008 < array_2008.length && sorting_2008) {
        int key_2008 = array_2008[i_2008];
        int j_2008 = i_2008 - 1;

        StringBuilder stepLog_2008 = new StringBuilder();
        stepLog_2008.append("Langkah ").append(stepCount_2008)
            .append(": Memasukkan ").append(key_2008).append("\n");

        while (j_2008 >= 0 && array_2008[j_2008] > key_2008) {
            array_2008[j_2008 + 1] = array_2008[j_2008];
            j_2008--;
        }
        array_2008[j_2008 + 1] = key_2008;

        updateLabels_2008();
        stepLog_2008.append("Hasil: ").append(arrayToString_2008(array_2008)).append("\n\n");
        stepArea_2008.append(stepLog_2008.toString());

        i_2008++;
        stepCount_2008++;

        if (i_2008 == array_2008.length) {
            sorting_2008 = false;
            stepButton_2008.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }
}

```

Gambar 2.10

```

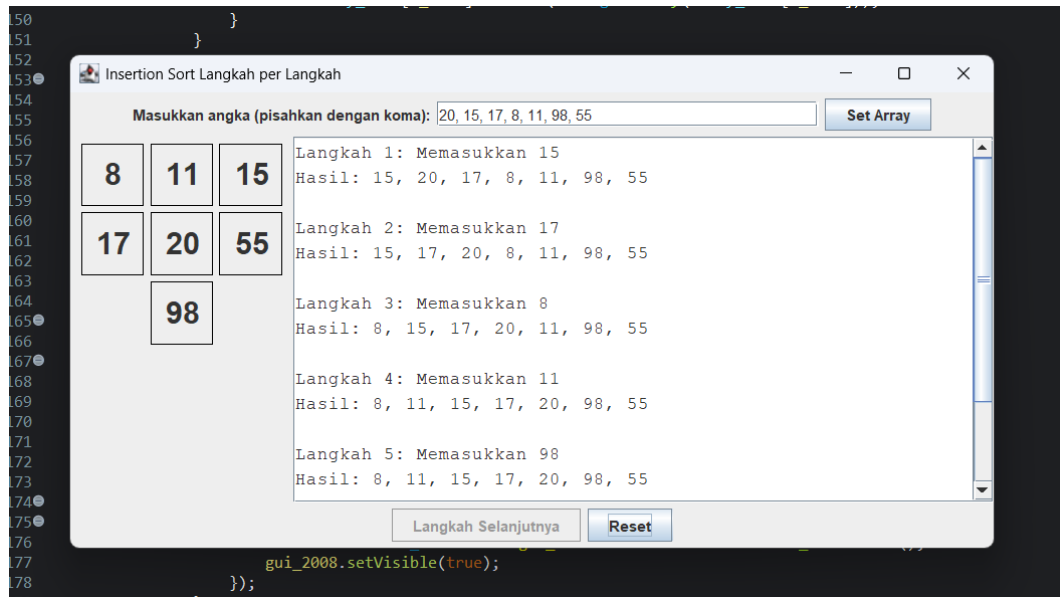
private void reset_2008() {
    inputField_2008.setText("");
    panelArray_2008.removeAll();
    panelArray_2008.revalidate();
    panelArray_2008.repaint();
    stepArea_2008.setText("");
    stepButton_2008.setEnabled(false);
    sorting_2008 = false;
    i_2008 = 1;
    stepCount_2008 = 1;
}

private String arrayToString_2008(int[] arr_2008) {
    StringBuilder sb_2008 = new StringBuilder();
    for (int k_2008 = 0; k_2008 < arr_2008.length; k_2008++) {
        sb_2008.append(arr_2008[k_2008]);
        if (k_2008 < arr_2008.length - 1) sb_2008.append(", ");
    }
    return sb_2008.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511532008 gui_2008 = new InsertionSortGUI_2511532008();
        gui_2008.setVisible(true);
    });
}

```

Gambar 2.11



Gambar 2.12

BAB III

PENUTUP

Berdasarkan hasil implementasi dan pengujian terhadap keempat kode program yang telah dibuat, dapat disimpulkan bahwa masing-masing algoritma pengurutan memiliki karakteristik perpindahan data yang unik di dalam memori; Bubble Sort mengandalkan pertukaran elemen berturut-turut hingga nilai terbesar mengapung ke ujung kanan, Insertion Sort menyisipkan elemen ke posisi yang tepat dengan menggeser bagian yang sudah terurut, sedangkan Selection Sort meminimalkan operasi penukaran data dengan hanya menukar satu kali setelah menemukan nilai minimum di setiap putaran. Selain itu, pengembangan aplikasi berbasis Java Swing seperti InsertionSortGUI membuktikan bahwa konsep algoritma yang abstrak dapat ditransformasikan menjadi alat bantu visual yang interaktif dan dinamis. Melalui visualisasi langkah demi langkah (step-by-step), proses pergeseran indeks array dan penempatan angka kunci (key) menjadi jauh lebih mudah dipahami secara intuitif dibandingkan hanya melihat keluaran teks statis pada console biasa.

TUGAS PRAKTIKUM

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;

class Mahasiswa_2511532008 {
    private String nama_2008;
    private String nim_2008;
    private String prodi_2008;

    public Mahasiswa_2511532008(String nama_2008, String nim_2008, String prodi_2008) {
        this.nama_2008 = nama_2008;
        this.nim_2008 = nim_2008;
        this.prodi_2008 = prodi_2008;
    }

    public String getNama_2008() {
        return nama_2008;
    }

    public void setNama_2008(String nama_2008) {
        this.nama_2008 = nama_2008;
    }

    public String getNim_2008() {
        return nim_2008;
    }

    public void setNim_2008(String nim_2008) {
        this.nim_2008 = nim_2008;
    }

    public String getProdi_2008() {
        return prodi_2008;
    }

    public void setProdi_2008(String prodi_2008) {
        this.prodi_2008 = prodi_2008;
    }

    public String toString_2008() {
        return nama_2008;
    }
}
```

```
public class AplikasiSortMahasiswa_2511532008 extends JFrame {
    private JTextField txtNama_2008, txtNim_2008, txtProdi_2008;
    private JButton btnTambah_2008, btnHapus_2008, btnSort_2008;
    private JComboBox<String> cmbAlgo_2008;
    private JTextArea txtAreaLog_2008;
    private JTable table_2008;
    private DefaultTableModel tableModel_2008;
    private ArrayList<Mahasiswa_2511532008> listMahasiswa_2008;

    public AplikasiSortMahasiswa_2511532008() {
        listMahasiswa_2008 = new ArrayList<>();
        initGui_2008();
    }
}
```

```

private void tambahData_2008() {
    String nama_2008 = txtNama_2008.getText().trim();
    String nim_2008 = txtNim_2008.getText().trim();
    String prodi_2008 = txtProdi_2008.getText().trim();

    if (nama_2008.isEmpty() || nim_2008.isEmpty() || prodi_2008.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Semua field harus diisi!", "Peringatan", JOptionPane.WARNING_MESSAGE);
        return;
    }

    Mahasiswa_2511532008 mhs_2008 = new Mahasiswa_2511532008(nama_2008, nim_2008, prodi_2008);
    listMahasiswa_2008.add(mhs_2008);

    tableModel_2008.addRow(new Object[]{mhs_2008.getNim_2008(), mhs_2008.getNama_2008(), mhs_2008.getProdi_2008()});

    txtNama_2008.setText("");
    txtNim_2008.setText("");
    txtProdi_2008.setText("");
    txtNama_2008.requestFocus();
}

private void hapusData_2008() {
    int barisTerpilih_2008 = table_2008.getSelectedRow();
    if (barisTerpilih_2008 != -1) {
        listMahasiswa_2008.remove(barisTerpilih_2008);
        tableModel_2008.removeRow(barisTerpilih_2008);
    } else {
        JOptionPane.showMessageDialog(this, "Pilih baris tabel yang ingin dihapus!", "Peringatan", JOptionPane.WARNING_MESSAGE);
    }
}

private void jalankanSorting_2008() {
    if (listMahasiswa_2008.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Data mahasiswa kosong! Input data terlebih dahulu.", "Informasi", JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    ArrayList<Mahasiswa_2511532008> listKerja_2008 = new ArrayList<>(listMahasiswa_2008);
    txtAreaLog_2008.setText("");

    String algoTerpilih_2008 = (String) cmbAlgo_2008.getSelectedItem();

    if ("Insertion Sort".equals(algoTerpilih_2008)) {
        insertionSort_2008(listKerja_2008);
    } else if ("Selection Sort".equals(algoTerpilih_2008)) {
        selectionSort_2008(listKerja_2008);
    } else if ("Bubble Sort".equals(algoTerpilih_2008)) {
        bubbleSort_2008(listKerja_2008);
    }

    tableModel_2008.setRowCount(0);
    for (Mahasiswa_2511532008 mhs_2008 : listKerja_2008) {
        tableModel_2008.addRow(new Object[]{mhs_2008.getNim_2008(), mhs_2008.getNama_2008(), mhs_2008.getProdi_2008()});
    }
}

```

```

private void insertionSort_2008(ArrayList<Mahasiswa_2511532008> list_2008) {
    int n_2008 = list_2008.size();
    cetakLog_2008("=== INSERTION SORT ===");
    cetakLog_2008("Data Awal : " + formatArrayNama_2008(list_2008));

    for (int i_2008 = 1; i_2008 < n_2008; i_2008++) {
        Mahasiswa_2511532008 key_2008 = list_2008.get(i_2008);
        int j_2008 = i_2008 - 1;

        while (j_2008 >= 0 && list_2008.get(j_2008).getNama_2008().compareToIgnoreCase(key_2008.getNama_2008()) > 0) {
            list_2008.set(j_2008 + 1, list_2008.get(j_2008));
            j_2008--;
        }
        list_2008.set(j_2008 + 1, key_2008);
        cetakLog_2008("Langkah " + i_2008 + " : " + formatArrayNama_2008(list_2008));
    }
    cetakLog_2008("Hasil Akhir: " + formatArrayNama_2008(list_2008) + "\n");
}

private void selectionSort_2008(ArrayList<Mahasiswa_2511532008> list_2008) {
    int n_2008 = list_2008.size();
    cetakLog_2008("=== SELECTION SORT ===");
    cetakLog_2008("Data Awal : " + formatArrayNama_2008(list_2008));

    for (int i_2008 = 0; i_2008 < n_2008 - 1; i_2008++) {
        int minIdx_2008 = i_2008;
        for (int j_2008 = i_2008 + 1; j_2008 < n_2008; j_2008++) {
            if (list_2008.get(j_2008).getNama_2008().compareToIgnoreCase(list_2008.get(minIdx_2008).getNama_2008()) < 0) {
                minIdx_2008 = j_2008;
            }
        }

        Mahasiswa_2511532008 temp_2008 = list_2008.get(minIdx_2008);
        list_2008.set(minIdx_2008, list_2008.get(i_2008));
        list_2008.set(i_2008, temp_2008);

        cetakLog_2008("Pass " + (i_2008 + 1) + " : " + formatArrayNama_2008(list_2008));
    }
    cetakLog_2008("Hasil Akhir: " + formatArrayNama_2008(list_2008) + "\n");
}

```

```

private void bubbleSort_2008(ArrayList<Mahasiswa_2511532008> list_2008) {
    int n_2008 = list_2008.size();
    cetakLog_2008("=== BUBBLE SORT ===");
    cetakLog_2008("Data Awal : " + formatArrayNama_2008(list_2008));

    for (int i_2008 = 0; i_2008 < n_2008 - 1; i_2008++) {
        for (int j_2008 = 0; j_2008 < n_2008 - i_2008 - 1; j_2008++) {
            if (list_2008.get(j_2008).getNama_2008().compareToIgnoreCase(list_2008.get(j_2008 + 1).getNama_2008()) > 0) {
                Mahasiswa_2511532008 temp_2008 = list_2008.get(j_2008);
                list_2008.set(j_2008, list_2008.get(j_2008 + 1));
                list_2008.set(j_2008 + 1, temp_2008);
            }
        }
        cetakLog_2008("Pass " + (i_2008 + 1) + " : " + formatArrayNama_2008(list_2008));
    }
    cetakLog_2008("Hasil Akhir: " + formatArrayNama_2008(list_2008) + "\n");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new AplikasiSortMahasiswa_2511532008().setVisible(true);
    });
}
}

```

Program Pengurutan Nama Mahasiswa - NIM Suffix _2008

Form Input Data Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

NIM	Nama Mahasiswa	Program Studi
2511539001	Asep	Informatika
2511539004	Bambang	Informatika
2511539005	Cecep	Informatika
2511539002	Dono	Informatika
2511539003	Eko	Informatika

Visualisasi Proses Sorting

Pilih Algoritma:

```

=== SELECTION SORT ===
Data Awal : [Asep, Dono, Eko, Bambang, Cecep]
Pass 1 : [Asep, Dono, Eko, Bambang, Cecep]
Pass 2 : [Asep, Bambang, Eko, Dono, Cecep]
Pass 3 : [Asep, Bambang, Cecep, Dono, Eko]
Pass 4 : [Asep, Bambang, Cecep, Dono, Eko]
Hasil Akhir: [Asep, Bambang, Cecep, Dono, Eko]

```

Program Pengurutan Nama Mahasiswa - NIM Suffix _2008

Form Input Data Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

NIM	Nama Mahasiswa	Program Studi
2511539001	Asep	Informatika
2511539004	Bambang	Informatika
2511539005	Cecep	Informatika
2511539002	Dono	Informatika
2511539003	Eko	Informatika

Visualisasi Proses Sorting

Pilih Algoritma:

```

=== INSERTION SORT ===
Data Awal : [Asep, Dono, Eko, Bambang, Cecep]
Langkah 1 : [Asep, Dono, Eko, Bambang, Cecep]
Langkah 2 : [Asep, Dono, Eko, Bambang, Cecep]
Langkah 3 : [Asep, Bambang, Dono, Eko, Cecep]
Langkah 4 : [Asep, Bambang, Cecep, Dono, Eko]
Hasil Akhir: [Asep, Bambang, Cecep, Dono, Eko]

```

Program Pengurutan Nama Mahasiswa - NIM Suffix _2008

Form Input Data Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

NIM	Nama Mahasiswa	Program Studi
2511539001	Asep	Informatika
2511539004	Bambang	Informatika
2511539005	Cecep	Informatika
2511539002	Dono	Informatika
2511539003	Eko	Informatika

Visualisasi Proses Sorting

Pilih Algoritma:

```

=== BUBBLE SORT ===
Data Awal : [Asep, Dono, Eko, Bambang, Cecep]
Pass 1 : [Asep, Dono, Bambang, Cecep, Eko]
Pass 2 : [Asep, Bambang, Cecep, Dono, Eko]
Pass 3 : [Asep, Bambang, Cecep, Dono, Eko]
Pass 4 : [Asep, Bambang, Cecep, Dono, Eko]
Hasil Akhir: [Asep, Bambang, Cecep, Dono, Eko]

```